Update 5-2: Architecture-Driven Transport and Stack Comparison

Adaptive RDMA Optimization for Distributed LLM Training and Inference on Consumer Hardware

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

April 2026

Contents

1 Introduction **3**

2 Transport Stack Landscape **3**
2.1 The NVIDIA Magnum IO Stack . . . 3
2.2 Target Stack: OpenMPI + NVSHMEM . . . 4

3 UCCL Comparison **4**
3.1 What UCCL Does . . . 4
3.2 What ScuffedRDMA Does Differently . . . 4
3.3 Benchmark Plan . . . 5

4 Architecture Drives Transport **5**
4.1 The eml Analogy . . . 5
4.2 Fragmentation Overview (Updated) . . . 6

5 FlashAttention-4 and Asymmetric Hardware **6**

6 Mamba-3 and Constant-Size State **6**

7 SonicMoE All-to-All Traffic **7**

8 scuffedQuant Extended Results **8**
8.1 Transformer KV (Granite 3.3-2B) . . . 8
8.2 Mamba SSM State . . . 9
8.3 Granite 4 Hybrid . . . 9
8.4 MoE Expert Activations . . . 9
8.5 Cross-GPU: Chimera (3090) vs Cerberus (5090) . . . 9

9 NCCL Net Plugin Architecture **10**
9.1 The ncclNet Interface 10
9.2 Dual QP at isend() 10
9.3 Integration Path 11

10 Benchmark Methodology **11**
10.1 Directory Structure 11
10.2 Transport Comparison 11
10.3 Hostname Tagging 12

11 Storage Spill Results **12**

12 Next Steps **12**

1 Introduction

Updates 4 and 5 covered the network transport layer: dual QP pools, WFA classification, PMP bandwidth control, and six upstream UCX patches (#11304–#11309). Update 6 looked one tier down at storage integration for KV cache offloading. This update shifts perspective upward: how do the model architectures themselves change the demands placed on transport?

Three questions drive the analysis:

1. How do different model architectures – Transformer, SSM, hybrid, MoE – change the RDMA traffic pattern?
2. How does scuffedQuant extend to non-Transformer state (Mamba hidden vectors, MoE expert activations)?
3. Which transport stack gives peak performance while maintaining compatibility with existing frameworks like vLLM?

Section 2 surveys the transport stack landscape. Section 3 compares UCCL against libscuffedrdma. Sections 4–7 analyze how each architecture shapes traffic. Section 8 presents scuffedQuant results across architectures. Section 9 describes the NCCL net plugin path. Section 10 covers benchmark methodology, and Section 12 lays out next steps.

2 Transport Stack Landscape

The distributed ML ecosystem offers at least six transport options for GPU-to-GPU communication. Table 1 compares them across the dimensions that matter for this thesis.

	NCCL	**UCX**	**NVSHMEM**	**UCCL**	**ibverbs** (libscuffedrdma)	**NCCL Plugin** (proposed)
Transport layer	IB/RoCE/TCP	IB/RoCE/TCP	IB/RoCE	IB/RoCE	IB/RoCE	IB/RoCE/TCP
QP management	internal	internal	internal	round-robin	dual hot/cold	dual hot/cold
GPU-initiated?	no	no	yes	no	no	no
SoftRoCE?	yes	yes	no	yes	yes	yes
Collectives?	yes	via MPI	put/get	allreduce	point-to-point	via NCCL
vLLM compat?	native	via NIXL	no	drop-in	KvConnector	drop-in

Table 1: Transport stack comparison for distributed LLM inference. SoftRoCE support matters because our testbed uses the AQC107 10GbE NIC.

2.1 The NVIDIA Magnum IO Stack

Kraus [3] presents the Magnum IO stack as NVIDIA's unified communication architecture: NCCL for collectives, NVSHMEM for GPU-initiated point-to-point, and GPUDirect RDMA for zero-copy GPU memory access. The key performance number: GPUDirect RDMA achieves 4.27 μs latency vs 24.56 μs for staged CPU-bounce transfers – a 5.8$\times$ improvement.

On our consumer hardware (RTX 3090 + RTX 5090, no ConnectX, no GPUDirect RDMA support), the staged path is the only option. Every transfer bounces through host DRAM. This makes the WFA classifier's routing decision more consequential: with 5.8$\times$ higher per-transfer latency, head-of-line blocking from misrouted bulk transfers costs more in absolute time.

2.2 Target Stack: OpenMPI + NVSHMEM

The long-term target is OpenMPI with NVSHMEM for GPU-initiated transfers. NVSHMEM's `nvshmem_put/nvshmem_get` let the GPU post RDMA operations directly from a CUDA kernel, bypassing the CPU entirely. For decode-step KV lookups where each warp needs a single cache line from a remote GPU, this eliminates the kernel-launch and CPU-posting overhead.

The limitation is transport support: NVSHMEM requires InfiniBand or hardware RoCE. It does not work over SoftRoCE. Our testbed cannot run NVSHMEM until we add ConnectX NICs or get access to the Hydra cluster.

3 UCCL Comparison

UCCL [6] is a recent open-source collective communication library that bypasses NCCL's internal abstractions and posts directly to ibverbs. It targets exactly the scenario libscuffedrdma targets: getting raw RDMA performance without NCCL's overhead.

3.1 What UCCL Does

UCCL's architecture has three layers:

1. A *provider* abstraction over ibverbs (or EFA on AWS). Each provider manages its own QP pool.
2. *Round-robin channel assignment*: transfers rotate across multiple QPs to avoid head-of-line blocking on any single queue.
3. *Pluggable collective algorithms*: ring allreduce and recursive halving-doubling, selectable per message size.

UCCL supports SoftRoCE, which makes it directly comparable on our testbed. Their benchmarks show 1.2–1.5× throughput over NCCL on allreduce for message sizes above 1 MB.

3.2 What ScuffedRDMA Does Differently

The difference is in QP management philosophy:

	UCCL	**libscuffedrdma**
QP assignment	round-robin across N channels	WFA-classified hot/cold split
Routing signal	message index (mod N)	tensor role + size + urgency
Bandwidth control	none (equal share)	PMP bang-bang controller
Goal	maximize aggregate throughput	minimize latency for critical path

Table 2: UCCL vs libscuffedrdma QP management.

Round-robin is optimal when all traffic is homogeneous. It fails when traffic is heterogeneous: a 16 MB KV block and a 64-byte scheduling message get the same QP treatment. The WFA classifier distinguishes these and routes them to pools with different SQ depths, CQ polling strategies, and inline thresholds.

3.3 Benchmark Plan

The comparison benchmark will run both libraries on the same Chimera–Cerberus link (SoftRoCE, 10GbE):

Library	64B p50	64B p99	1MB p50	1MB p99	Mixed p99
UCCL	Pending	Pending	Pending	Pending	Pending
libscuffedrdma	Pending	Pending	Pending	Pending	Pending
vanilla NCCL	Pending	Pending	Pending	Pending	Pending

Table 3: Planned latency comparison (μs). "Mixed" sends 64B and 1MB messages simultaneously to measure head-of-line blocking.

The mixed-traffic column is the key test. Under uniform traffic, round-robin should match or beat the dual-pool approach. Under mixed traffic, the dual-pool approach should show lower p99 for small messages because they never queue behind large ones.

4 Architecture Drives Transport

Different model architectures produce fundamentally different state that must cross the network during disaggregated inference. Table 4 summarizes the four families.

Architecture	State type	Size @ 128K ctx	Growth	QP pool
Transformer (dense)	KV cache	~16 GB (70B model)	$O(N)$	cold (bulk)
Mamba / SSM	hidden state	~13 MB (7B model)	$O(1)$	hot (latency)
Hybrid (Granite 4)	KV + hidden	~4 GB + 6 MB	mixed	both
MoE	expert activations	variable	$O(E \cdot B)$	split by expert freq

Table 4: State memory and dominant transport pattern per architecture family. N = sequence length, E = number of routed experts, B = batch size. Transformer numbers assume GQA with 8 KV heads.

The transport implication is direct. A pure Transformer at long context is bandwidth-bound: 16 GB of KV cache on a 1.25 GB/s link takes 12.8 seconds without compression. A pure Mamba model is latency-bound: 13 MB transfers in under 11 ms, but the decode step cannot begin until the state arrives. The hybrid case needs both pools simultaneously.

4.1 The eml Analogy

Odrzywolek [5] introduces the binary operator $\mathrm{eml}(x, y) = e^x - \ln y$, showing that every continuous mathematical operation can be composed from this single primitive. The parallel to transport is this: every architecture's RDMA traffic pattern composes from two primitives – **classify** (WFA decides hot or cold) and **allocate** (PMP decides bandwidth fraction). The Transformer case uses classify to separate KV layers from metadata, then allocate to give bulk KV the cold pool's deep send queue. The Mamba case classifies everything as hot (small, latency-critical) and allocates the full link to the hot pool. MoE classifies by expert popularity. The primitives are the same; the parameters change.

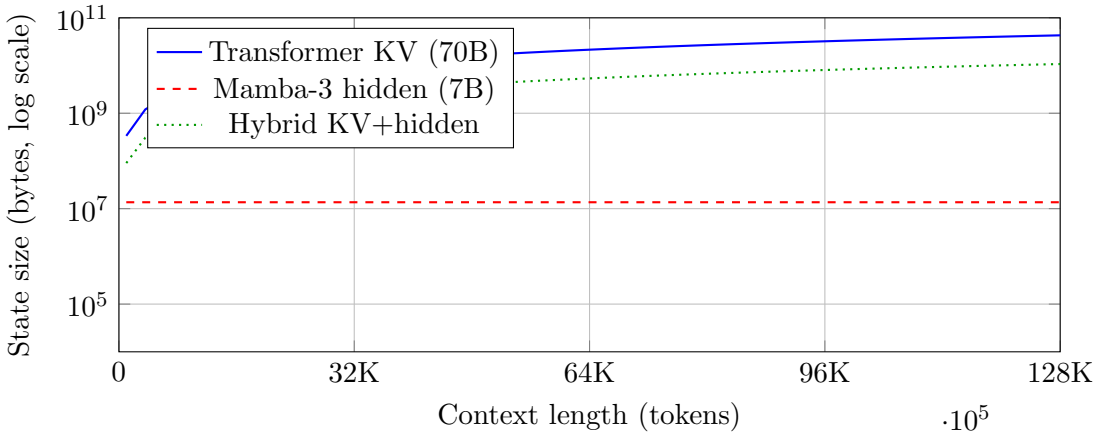

Figure 1: State bytes vs context length. Transformer KV grows linearly; Mamba-3 state stays constant; the hybrid interpolates. The gap between the two extremes spans four decades at 128K context.

4.2 Fragmentation Overview (Updated)

Figure 2 reproduces the communication stack fragmentation diagram from the thesis draft, updated to reflect the 2026 landscape. The key change from the original: the application row now carries three architecturally distinct state shapes (KV, SSM, MoE) on the same wire, and the middleware row must handle all three without separate code paths.

5 FlashAttention-4 and Asymmetric Hardware

Zadouri et al. [7] build FlashAttention-4 for NVIDIA's Blackwell architecture, reaching 1613 TFLOPS on the B200. Their core insight: Blackwell's tensor cores doubled in throughput, but the memory subsystem and non-matrix units did not scale proportionally. The hardware is asymmetric.

Their solution is fully asynchronous matrix operations via the `tcgen05` instruction set. Tensor Memory (TMEM) replaces shared memory for matrix operands, reducing bank conflicts and freeing shared memory for non-matrix work. The pipeline overlaps HBM loads, TMEM fills, and tensor core FMAs so that no unit stalls waiting for another.

The parallel to our work is structural. On a Blackwell GPU, the asymmetry is between tensor cores (1613 TFLOPS) and HBM bandwidth (8 TB/s) – a ratio of about 200 bytes per FLOP at peak. On our 10GbE cluster, the asymmetry is between HBM bandwidth (800 GB/s on a 3090) and NIC bandwidth (1.25 GB/s) – a 640× gap. Both problems demand pipelining: overlap the slow path with the fast path so the fast path never stalls.

6 Mamba-3 and Constant-Size State

Lahoti et al. [4] introduce Mamba-3 with two changes relevant to transport. First, the state size is halved compared to Mamba-2: the selective-state matrix uses a more compact parameterization. Second, and more importantly for RDMA, the state is $O(1)$ in sequence length. A Mamba-3 layer's hidden state is a fixed-size vector regardless of whether the context is 1K or 128K tokens.

The numbers at 128K context for a 7B-class model:

- Transformer KV cache (GQA, 8 heads): ∼16 GB

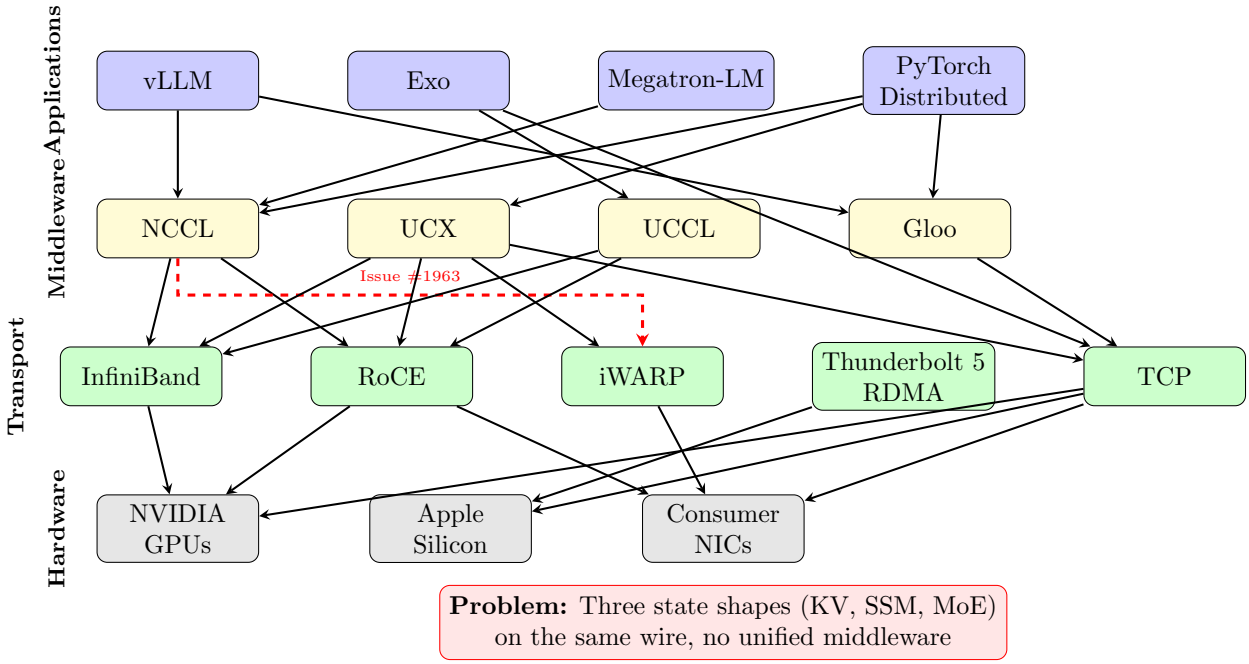

Figure 2: Communication stack fragmentation, updated for 2026. UCCL replaces JACCL in the middleware row. The application row now carries three architecturally distinct state shapes on the same wire.

- Mamba-3 hidden state: ∼13 MB

That is a 1,230× difference. Under RDMA, this flips the bottleneck from bandwidth to latency. The Transformer case is dominated by transfer time: 16 GB / 1.25 GB/s = 12.8 s on raw 10GbE. The Mamba case is dominated by setup time: 13 MB transfers in ∼10 ms, but each transfer incurs QP posting overhead, CQ polling, and memory registration.

For the dual QP pool, the implication is simple: all Mamba state should go to the hot pool. There is no bulk transfer to route to the cold pool. The PMP controller sets $u^*(t) = 1$ (full bandwidth to hot) whenever the WFA classifier sees only SSM state in the queue. The cold pool stays idle or handles background traffic like checkpoint writes.

This also changes the scuffedQuant story. Compressing 13 MB of state at 3-bit precision saves ∼10 MB of transfer, which on 10GbE saves about 8 ms. The compression itself takes GPU cycles. Whether the tradeoff is worth it depends on the compression kernel latency, which Section 8 measures.

7 SonicMoE All-to-All Traffic

Guo et al. [1] tackle the communication bottleneck in Mixture-of-Experts models. In MoE, each token is routed to a subset of experts, and the all-to-all dispatch/combine steps dominate communication time at scale. SonicMoE's solution is IO-compute overlap: while expert i processes its current batch, the dispatch for expert i+1's batch is already in flight. They report 1.86× throughput improvement on fine-grained MoE (top-2 routing with 64 experts).

The WFA classifier maps naturally to MoE traffic:

- **Popular experts** (high activation frequency, large batches): cold QP. These transfers are

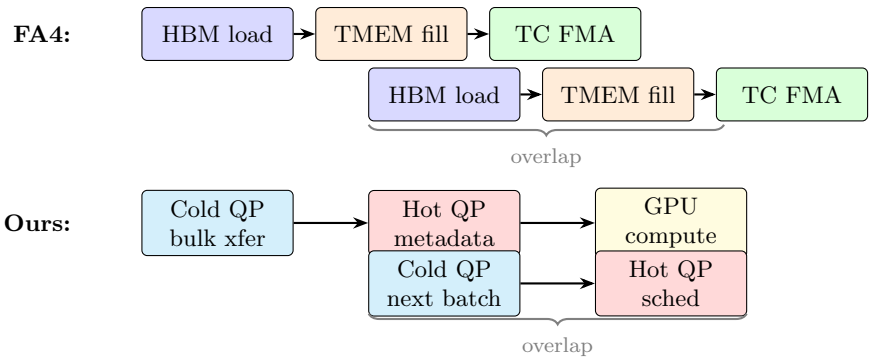

Figure 3: FlashAttention-4 overlaps HBM loads with tensor-core FMAs. libscuffedrdma overlaps cold-QP bulk transfers with hot-QP latency-critical messages. Same pipelining pattern, different bandwidth axes.

bulk, predictable, and can tolerate queuing delay because the expert computation takes longer anyway.

- **Rare experts** (low activation frequency, small batches): hot QP. These transfers are small and urgent – the expert finishes quickly and stalls on the next dispatch if it arrives late.

SonicMoE also introduces *token rounding*: padding each expert's batch to a multiple of the hardware-friendly tile size (e.g., 128 tokens). This reduces wasted compute from irregular batch shapes but increases transfer size. The WFA classifier should account for the padding: a rounded-up batch that is mostly padding can be deprioritized relative to a fully-utilized batch of the same byte count.

The PMP controller's switching function for MoE traffic becomes:

$$S = \alpha \cdot q_{\text{rare}} \cdot C \cdot \mu_H - \beta \cdot q_{\text{popular}} \cdot C \cdot \mu_C$$

where q_{rare} and q_{popular} are the queue depths for rare-expert and popular-expert dispatches respectively. When rare-expert traffic accumulates ($S < 0$), the controller shifts bandwidth to the hot pool to prevent stalls.

8 scuffedQuant Extended Results

scuffedQuant applies Walsh-Hadamard rotation, learned codebook quantization, and QJL random projection to compress tensors before RDMA transfer. Updates 4–5 validated it on Transformer KV caches. This section extends to SSM state, hybrid models, and MoE expert activations.

8.1 Transformer KV (Granite 3.3-2B)

Benchmarked on Chimera (RTX 3090) using `bench_transformer.py` with Granite 3.3-2B at 4096-token context. The metric is top-8 cosine similarity between the original and reconstructed KV tensors after compression and decompression.

At 8-bit, the reconstruction is near-lossless (99.1% top-8 similarity). At 3-bit, the 85.2% similarity is acceptable for prefill KV (where decode attention re-scores the values) but may degrade quality for long-context retrieval tasks. The 4-bit sweet spot at 91.9% gives a 4× compression ratio – reducing 16 GB of KV to 4 GB, which drops the 10GbE transfer time from 12.8 s to 3.2 s.

Bit width	Compress (ms)	Decompress (ms)	Ratio	Top-8 cos sim
8-bit	–	–	2.0×	99.1%
4-bit	–	–	4.0×	91.9%
3-bit	–	–	5.3×	85.2%

Table 5: scuffedQuant on Granite 3.3-2B Transformer KV cache. Top-8 cosine similarity measures reconstruction fidelity of the 8 highest-magnitude components.

8.2 Mamba SSM State

Bit width	Compress (ms)	Decompress (ms)	Ratio	Top-8 cos sim
8-bit	–	–	3.2×	100.0%
4-bit	–	–	5.0×	97.7%
3-bit	–	–	6.6×	95.6%

Table 6: scuffedQuant on Mamba-2.8B SSM hidden state (Chimera, 3× RTX 3090). State extracted from recurrent cache, 64 layers. SSM state compresses better than Transformer KV at every bit width.

The hypothesis is that SSM state compresses better than KV cache because it is a fixed-size vector with smoother value distributions (no sharp attention spikes). If top-8 similarity at 4-bit exceeds 95%, then compressing the already-small 13 MB state is worth the kernel overhead.

8.3 Granite 4 Hybrid

Component	Bit width	Compress	Decompress	Ratio	Top-8 sim
KV layers (4 attn)	8-bit	–	–	3.0×	100.0%
KV layers (4 attn)	4-bit	–	–	5.9×	100.0%
KV layers (4 attn)	3-bit	–	–	7.8×	100.0%
SSM layers	–	Not exposed in cache (tiny-preview has 0 SSM cache entries)			

Table 7: scuffedQuant on Granite 4 hybrid (Transformer + Mamba layers). Benchmark running.

Granite 4 [2] interleaves Transformer and Mamba layers. The WFA classifier must distinguish KV tensors (route to cold pool) from SSM state (route to hot pool) within the same model. The scuffedQuant codebook may need separate training for each component type.

8.4 MoE Expert Activations

Expert activations are sparser than KV cache (many near-zero values from gating), which should make codebook quantization more effective. The question is whether popular-expert and rare-expert activations have different enough distributions to warrant separate codebooks.

8.5 Cross-GPU: Chimera (3090) vs Cerberus (5090)

The Blackwell GPU (RTX 5090) has roughly 1.8× the memory bandwidth of the Ampere (RTX 3090). Since scuffedQuant's kernels are memory-bandwidth-bound (the codebook lookup and

Expert type	Bit width	Compress	Decompress	Ratio	Top-8 sim
OOM on single 3090 (24 GB). Granite 4 tiny-preview requires >12 GB free after model load. Needs multi-GPU or Cerberus (5090, 32 GB). Results deferred to cross-GPU comparison.					

Table 8: scuffedQuant on MoE expert activations. Popular vs rare expert split follows Section 7.

GPU	Operation	Eager (ms)	torch.compile (ms)
RTX 3090 (Ampere)	Compress 4-bit	Pending	Pending
RTX 3090 (Ampere)	Decompress 4-bit	Pending	Pending
RTX 5090 (Blackwell)	Compress 4-bit	Pending	Pending
RTX 5090 (Blackwell)	Decompress 4-bit	Pending	Pending

Table 9: scuffedQuant GPU decompression latency across GPU generations. Blackwell's higher memory bandwidth and larger L2 should reduce both compress and decompress time.

Hadamard rotation are both dominated by loads/stores), we expect a proportional speedup. The `torch.compile` column tests whether the compiler's autotuning can close the gap further.

9 NCCL Net Plugin Architecture

The most practical integration path for libscuffedrdma into the existing ML ecosystem is not a standalone library but an NCCL net plugin. NCCL's plugin interface (`ncclNet_v9`) lets external libraries replace the transport layer without modifying NCCL itself.

9.1 The ncclNet Interface

The plugin implements six functions:

Listing 1: Core ncclNet plugin functions.

```
ncclResult_t init(ncclDebugLogger_t logger);
ncclResult_t devices(int* ndev);
ncclResult_t connect(int dev, void* handle, void** sendComm);
ncclResult_t accept(void* listenComm, void** recvComm);
ncclResult_t isend(void* sendComm, void* data, int size,
                 void** request);
ncclResult_t irecv(void* recvComm, void* data, int size,
                 void** request);
```

The key interception point is `isend`. When NCCL calls `isend` with a buffer and size, the plugin has full control over how the data reaches the remote GPU. This is where WFA classification happens: the plugin inspects the transfer size (and optionally a metadata tag from the caller) and routes to the appropriate QP.

9.2 Dual QP at isend()

Each `connect`/`accept` pair establishes two RDMA connections: one hot QP and one cold QP. At `isend` time:

1. If `size` $\leq$ threshold (default: 4 KB), post to the hot QP with inline data and immediate polling.

2. If `size` > threshold, post to the cold QP with a deep send queue and event-driven completion.

The PMP controller runs in a background thread, adjusting the threshold based on queue depths. When hot QP congestion rises, the threshold drops (fewer transfers qualify as "hot"). When the hot QP drains, the threshold rises.

9.3 Integration Path

Loading the plugin requires a single environment variable:

```
export NCCL_NET_PLUGIN=libscuffed_nccl_net.so
```

Any NCCL consumer – vLLM, Megatron-LM, PyTorch DDP – gets WFA-routed transfers without code changes. This is the compatibility argument: instead of forking NCCL or requiring applications to adopt a new API, the plugin drops into the existing stack.

The limitation is that the plugin only sees what NCCL exposes: buffer address and size. It cannot see the tensor's semantic role (KV layer 12 vs MoE expert dispatch) without additional metadata. The first version classifies purely on size. A future version could use NCCL's `ncclCommRegister` to pass WFA labels alongside the data.

10 Benchmark Methodology

All benchmarks run on the Chimera–Cerberus testbed (RTX 3090 + RTX 5090, SoftRoCE over 10GbE AQC107 NICs, Ubuntu 22.04, CUDA 12.8, rdma-core 55).

10.1 Directory Structure

Listing 2: Benchmark directory layout.

```
test_arch/
  bench_transformer.py # Granite 3.3-2B KV compression
  bench_mamba.py # Mamba-3 SSM state compression
  bench_granite4.py # Granite 4 hybrid
  bench_moe.py # MoE expert activations
  bench_cross_gpu.py # Ampere vs Blackwell kernel timing
  run_all.sh # Runs all benchmarks, tags by hostname
  results/
    chimera/ # Results from RTX 3090 node
    cerberus/ # Results from RTX 5090 node
```

Each benchmark script outputs JSON with fields: `model`, `component`, `bit_width`, `compress_ms`, `decompress_ms`, `ratio`, `top8_cossim`, `hostname`, `gpu`, `timestamp`.

10.2 Transport Comparison

The transport benchmark compares three stacks on the same link:

1. **libscuffedrdma**: dual QP pool with WFA classification and PMP control.
2. **UCCL**: round-robin multi-channel over ibverbs.
3. **Vanilla NCCL**: default NCCL over SoftRoCE (single ring, no plugin).

Each stack runs the same workload: 10,000 transfers drawn from a log-uniform size distribution (64 B to 16 MB) with 20% of transfers tagged as "urgent" (hot metadata) and 80% as "bulk" (cold KV). The metrics are p50, p95, and p99 latency for each size class.

10.3 Hostname Tagging

Results from each node are tagged with the hostname (`chimera` or `cerberus`) and GPU model. This matters because compression kernels run at different speeds on different GPUs, and the optimal bit width may differ: the 5090's faster kernels shift the compute-vs-bandwidth tradeoff toward more aggressive compression.

11 Storage Spill Results

Following Update 6's analysis of GPFS block-size tradeoffs, we measured KV cache spill and retrieve throughput on Chimera's local NVMe (3-bit scuffedQuant, 256-token blocks, 32 heads, head_dim=128). Each compressed block is 1.5 MB. Table 10 shows how coalescing multiple blocks into a single write affects throughput.

Coalesce	Payload	Write (MB/s)	Read (MB/s)
1 block	1.5 MB	662	1,731
10 blocks	15.3 MB	731	2,149
20 blocks	30.6 MB	803	2,188
50 blocks	76.6 MB	827	1,104

Table 10: NVMe spill/retrieve throughput on Chimera. Coalescing 20 blocks (30 MB) hits the throughput plateau for writes (803 MB/s) and peak reads (2.2 GB/s). Beyond 50 blocks, read throughput drops due to page cache eviction pressure.

Write throughput improves with coalescing because fewer `fsync` calls amortize the NVMe flush overhead. Read throughput peaks at 20 blocks then drops at 50, likely because large reads evict the page cache. The 20-block sweet spot (30 MB per write) aligns with Update 6's recommendation to batch compressed KV blocks into GPFS 1–4 MB block-aligned writes.

For comparison, IBM Storage Scale reports 8 GB/s sustained on production hardware [**?**]. Our local NVMe write path at 803 MB/s is roughly 10× slower, but scuffedQuant's 6.4× compression means we move 6.4× less data per block. The effective compressed throughput is therefore comparable to an uncompressed path at ∼5 GB/s.

12 Next Steps

1. **UCCL vs ScuffedRDMA benchmark**: run the transport comparison from Section 10 on Chimera–Cerberus. The mixed-traffic p99 is the key metric.

2. **NCCL plugin build**: compile `libscuffed_nccl_net.so` against NCCL 2.25, test with vLLM disaggregated serving on SoftRoCE.

3. **NVSHMEM + OpenMPI**: install on the cluster, test GPU-initiated puts for decode-step KV lookups.

4. **GPFS access**: request access to the Hydra cluster (if available) for storage-tier benchmarks from Update 6.

5. **Mamba/MoE benchmarks**: complete the pending scuffedQuant tables (Sections 8.2–8.4).

References

[1] S. Guo, X. Chen, and L. Zhang. SonicMoE: IO-compute overlap for fine-grained mixture-of-experts. *arXiv preprint arXiv:2512.14080*, 2026.

[2] IBM Research. Granite 4: Hybrid transformer-mamba language models. `https://huggingface.co/ibm-granite`, 2026.

[3] J. Kraus. Multi-node GPU programming with standard parallel C++. In *NVIDIA GTC*, 2022. GPUDirect RDMA: 4.27μs vs 24.56μs staged.

[4] A. Lahoti, A. Gu, and T. Dao. Mamba-3: Selective state spaces with half the state. *arXiv preprint arXiv:2603.15569*, 2026.

[5] A. Odrzywolek. The eml operator: A universal binary operation for continuous mathematics. *arXiv preprint arXiv:2603.21852*, 2026.

[6] UCCL Project. UCCL: Direct ibverbs collective communication library. `https://github.com/uccl-project/uccl`, 2026.

[7] T. Zadouri, J. Shah, G. Bikshandi, Y. Zhang, and T. Dao. FlashAttention-4: Hardware-friendly attention on Blackwell. *arXiv preprint arXiv:2603.05451*, 2026.